

CSCE 713 SOFTWARE SECURITY · MODULE 12 · LECTURE NOTES

Exploit Mitigations

What to do when the defect is already shipped.

MODULE OUTCOMES

- Explain each mitigation and what it stops.
- Explain how each has been bypassed.
- Explain the cost of each.
- Explain compiler hardening you should already enable.
- Assess a binary's mitigations.

1 The strategy

MITIGATIONS DO NOT FIX DEFECTS; THEY MAKE EXPLOITATION EXPENSIVE AND UNRELIABLE

Every mitigation in this module leaves the defect entirely present — the overflow still overflows, the stale pointer is still stale — and makes the step from having a defect to having control of execution substantially harder.

Which is worth a great deal in practice. An exploit that requires a separate information leak as a first stage, works one time in ten, and breaks on the next compiler release is a fundamentally different proposition from one that works reliably and indefinitely — both economically and operationally.

And it is worth being explicit about the arithmetic: mitigations shift effort from the defender to the attacker, which is the opposite direction to most of CSCE 701 Module 01's asymmetry and is why they have been worth the investment despite every one being bypassable.

But none of them is a substitute for not having the defect — and every single one has been bypassed in practice, which §2 covers explicitly rather than omitting, because a mitigation whose bypass you do not know is a mitigation you will over-trust.

2 The mitigations

Mitigation	What it stops	How it has been bypassed
Stack canary	A sequential stack overflow reaching the saved return address.	Leaking the canary value first; writing non-sequentially past it; overwriting a local pointer instead of the return address.

Mitigation	What it stops	How it has been bypassed
Non-executable memory (NX/DEP)	Executing shellcode the attacker injected as data.	Reusing code already present in the process — return-oriented programming.
Address space layout randomisation	Knowing the address of anything.	An information leak; insufficient entropy on 32-bit; partial pointer overwrite; a non-PIE binary left at a fixed address.
Control-flow integrity	Indirect calls and jumps to unintended targets.	Choosing a target within the permitted set; data-only attacks that corrupt values rather than pointers.
Shadow stack / return address protection	Corruption of the return address specifically.	Forward-edge attacks, which it does not cover at all — so it pairs with CFI rather than replacing it.
Allocator hardening	Reliable control of heap layout (Module 02 §3).	Patience, repeated attempts, and information leaks — it reduces reliability rather than preventing the technique.

THE HISTORICAL SEQUENCE IS INSTRUCTIVE

Non-executable memory made injected shellcode useless — so attackers stopped injecting code and started reusing code already present in the process, chaining short sequences ending in a return.

Return-oriented programming exists because of NX.

ASLR then made the addresses of that existing code unknown — so attackers added an information leak as a mandatory first stage, **which is precisely why an out-of-bounds *read* is now a valuable primitive rather than a minor disclosure** (Module 02 §2's second row, and Module 01 §3's severity reasoning).

Control-flow integrity then constrained indirect calls to plausible targets — so attackers moved to data-only attacks, corrupting a privilege flag, a length field, or a file path rather than any pointer, which CFI does not address because no control flow is violated.

Which is CSCE 701 Module 01's asymmetry running in reverse — each defence closed off one technique and the attacker needed only one remaining technique — and the cost went up substantially at each step, which is the whole justification for the sequence. **The defender did not win and the price rose by orders of magnitude**, and that is a real result.

3 Hardening you should already have

```
-D_FORTIFY_SOURCE=3      bounds-checked string and
                        memory functions wherever the
                        size is known at compile time
-fstack-protector-strong stack canaries
-fstack-clash-protection stack probing
-Wl,-z,relro,-z,now      read-only relocations, so the
                        GOT is not writable
-fPIE -pie              position independent, so that
                        ASLR applies to the main
                        binary and not only libraries
-fcf-protection          hardware CFI on x86
-D_GLIBCXX_ASSERTIONS    bounds-checked C++ containers
```

AND THE WARNINGS AS ERRORS

```
-Wall -Wextra -Werror
-Wformat=2 -Wconversion (the last catches
                        Module 03's truncations)
```

CHECK WHAT YOU SHIPPED, not what you configured --
run checksec or hardening-check on the actual
artefact.

Check the artefact rather than the build configuration. A flag set in one makefile, and a vendored library or a third-party dependency built without it, is the common case — and the resulting binary has the mitigation on some objects and not others, which for ASLR and RELRO means effectively not at all. Verifying the binary is the step people skip, and it takes one command.

- Most of the list above costs under a few percent, and several cost nothing measurable at all — which makes leaving them off a decision rather than a trade-off, and usually a decision nobody made deliberately.
- FORTIFY_SOURCE and the container assertions can cost more in allocation-heavy or string-heavy code, so measure on your own workload rather than accepting either the published figure or the folklore.
- Control-flow integrity costs real performance and code size, and requires whole-program or link-time visibility — which makes it a genuine engineering decision rather than a free flag.
- And PIE historically cost a little on 32-bit x86, because of register pressure, and costs essentially nothing now — so the objection you may read in older material no longer applies.
- So the honest position: enable the cheap set unconditionally, and measure the rest against your own workload. The cheap set is free security and is off by default in a great many build systems — which makes checking your own build a high-yield afternoon and one of the few genuinely easy wins in this course.

4 Perspective

MITIGATIONS BUY TIME; SAFE LANGUAGES AND TOOLING REMOVE THE CLASS

Enable every mitigation you can afford — it is cheap, it applies to code you cannot change or do not control, and it degrades the reliability of exploits against defects you do not yet know you have, which is the only defence that has that property.

And do not mistake that for a solution. The defect is still present, and a sufficiently motivated and resourced adversary gets through — the published record on this is unambiguous, and §2's third column is the evidence.

So the ordering is clear: eliminate the class where you can (Module 02 §4's safe languages, Module 10 §2's safe-by-default interfaces), find the instances with tooling (Modules 07 through 09), and mitigate whatever remains. Three layers, in that order of preference.

Which is the proportionate position — mitigations are the last layer rather than the strategy, and a programme that consists only of hardening flags has not started. Equally, a programme that has rewritten nothing and also not enabled the free flags has left money on the table, which is the more common failure.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Szekeres et al. — SoK: Eternal War in Memory (free)	§1 and §2 systematised — every mitigation, what it stops, and what bypasses it, in one paper.
Shacham — The Geometry of Innocent Flesh on the Bone (free)	§2's first bypass in the original — the paper that introduced return-oriented programming.
Debian and Red Hat hardening guides, and checksec (free)	§3's flags with the distribution's rationale , plus the tool to verify the artefact.
MIT 6.858's mitigation labs (free, OCW)	§2 practically , on targets built for it — which is Module 01 §4's constraint satisfied.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 State what each of six mitigations stops** and how each has been bypassed.
- 2 Explain why NX produced ROP**, and why ASLR made OOB reads valuable.
- 3 Explain what a data-only attack is** and why CFI does not stop it.

- 4 **Run checksec on a binary you ship** and report what is missing.
- 5 **Run it on your dependencies' libraries** and compare.
- 6 **Enable the cheap hardening set** and verify it on the artefact.
- 7 **Measure the performance cost** on your own workload.
- 8 **Enable -Wconversion** and report how many Module 03 truncations it finds.
- 9 **Demonstrate a stack canary stopping an overflow** on your own test program.
- 10 **Write your three-layer plan:** eliminate, find, mitigate.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What do mitigations do and not do?
- 2 Why is an unreliable exploit meaningfully different?
- 3 Name six mitigations, what each stops, and a bypass for each.
- 4 Why are they deployed in combination?
- 5 Give the historical sequence $\text{NX} \rightarrow \text{ROP} \rightarrow \text{ASLR} \rightarrow \text{leak} \rightarrow \text{CFI} \rightarrow \text{data-only}$.
- 6 Why is an out-of-bounds read now valuable?
- 7 Name six hardening flags and what each provides.
- 8 Why check the artefact rather than the configuration?
- 9 Which flags are free and which cost real performance?
- 10 Give the three-layer ordering and the proportionate position.